

Dynamic Global Services

Direction Technique

Engagement de Test d’Intrusion

Audit de sécurité applicative, référentiel OWASP Top 10

Périmètre : Portail de bienvenue DGS

Environnement local et isolé via Docker, aucune mise en ligne

Direction Technique, Programme d’intégration technique

Contents

1	Note de cadrage de la mission	4
1.1	Contexte de la mission	4
1.2	Cadre d'engagement et règles de déontologie	4
1.3	Livrables attendus	4
1.4	Phase de montée en compétence préalable, obligatoire	4
1.5	Mise en place de l'environnement d'audit	5
1.6	Outillage, installation d'OWASP ZAP	5
2	Le référentiel OWASP Top 10	6
3	A03, Injection SQL	7
3.1	Comprendre	7
3.2	Pratiquer	7
3.3	Analyser et corriger	7
4	A03, Cross Site Scripting, XSS	7
4.1	Comprendre	7
4.2	Pratiquer	7
4.3	Analyser et corriger	8
5	A01, Contrôles d'accès défaillants	9
5.1	Comprendre	9
5.2	Pratiquer	9
5.3	Analyser et corriger	9
6	A07, Identification et authentification de mauvaise qualité	9
6.1	Comprendre	9
6.2	Pratiquer	9
6.3	Analyser et corriger	9
7	A02, Défaillances cryptographiques	11
7.1	Comprendre	11
7.2	Pratiquer	11
7.3	Analyser et corriger	11
8	A05, Mauvaise configuration de sécurité	11
8.1	Comprendre	11

8.2	Pratiquer	11
8.3	Analyser et corriger	11
9	A06, Composants vulnérables et obsolètes	13
9.1	Comprendre	13
9.2	Pratiquer	13
9.3	Analyser et corriger	13
10	A04, Conception non sécurisée	13
10.1	Comprendre	13
10.2	Pratiquer	13
10.3	Analyser et corriger	13
11	A08, Manque d'intégrité des données et du logiciel	15
11.1	Comprendre	15
11.2	Pratiquer	15
11.3	Analyser et corriger	15
12	A10, Falsification de requête côté serveur, SSRF	15
12.1	Comprendre	15
12.2	Pratiquer	15
12.3	Analyser et corriger	15
13	A09, Carence des systèmes de contrôle et de journalisation	16
13.1	Comprendre	16
13.2	Pratiquer	16
13.3	Analyser et corriger	16
14	Scénario final, démonstration d'impact	17
14.1	Le scénario	17
14.2	Objectif à atteindre	17
14.3	Contenu attendu dans cette partie du rapport	17
14.4	Critère de réussite	17
15	Prolongements professionnels	18

1. Note de cadrage de la mission

1.1. Contexte de la mission

La Direction Technique te confie ta première mission d'audit. Tu intègres l'équipe sécurité et tu es chargé d'évaluer la robustesse du portail de bienvenue de DGS avant sa mise en service. Tu débutes, et ce document t'accompagne pas à pas : à chaque étape il t'explique ce que tu fais, pourquoi la faille existe, comment un attaquant l'exploiterait, et surtout comment la corriger. Tu mèneras ton audit selon le référentiel de référence du secteur, l'OWASP Top 10. Considère que le client interne, c'est la Direction Technique, et que ton travail sera restitué comme un véritable rapport d'audit.

1.2. Cadre d'engagement et règles de déontologie

Toute mission de test d'intrusion s'exerce dans un cadre contractuel et déontologique strict.

- Ton périmètre d'engagement se limite strictement à l'application fournie, en local sur ta machine. Sortir de ce périmètre n'est pas autorisé, et l'application ne doit jamais être déployée sur internet.
- On ne teste jamais un système sans mandat écrit du client. Tester un système hors mandat est une infraction pénale dans la plupart des pays. Le cadre légal n'est pas une formalité administrative, c'est ce qui distingue un professionnel d'un délinquant.
- L'objectif est de comprendre et de protéger. Chaque attaque que tu apprends doit te servir à mieux défendre.

1.3. Livrables attendus

Le livrable central de ta mission est un rapport d'audit détaillé. À chaque étape, tu rédiges un texte qui explique ce que tu as fait, ce que tu as observé et ce que tu en conclus. Ce rapport pèse autant que la technique : dans une mission réelle, un test d'intrusion se conclut toujours par un rapport écrit remis au client et une restitution orale. Le code corrigé sera versé sur ton dépôt GitHub, et ton rapport sera transmis via le formulaire prévu.

1.4. Phase de montée en compétence préalable, obligatoire

Avant de toucher à l'application, prends le temps de chercher et de rédiger, avec tes propres mots, des réponses aux questions suivantes. Cette recherche conditionne ta compréhension de toute la suite.

1. Qu'est-ce qu'un test d'intrusion, aussi appelé pentest ? Quelles sont ses grandes étapes, de la reconnaissance au rapport final ?
2. Quelle différence fais-tu entre un test en boîte noire, en boîte grise et en boîte blanche ?
3. Qu'est-ce que l'OWASP, et qu'est-ce que le Top 10 ? À quoi sert ce classement ?
4. Qu'est-ce qu'un outil comme OWASP ZAP, et à quoi sert un proxy d'interception ?

Ta synthèse de recherche, une demi-page minimum :

1.5. Mise en place de l'environnement d'audit

- Dans le dossier du TP, lance `docker compose up -d -build`.
- Ouvre `http://localhost:8080`, tu dois voir le portail de bienvenue.
- Comptes de test fournis pour t'aider, `stagiaire` avec le mot de passe `password`, et `sophie` avec `sophie2024`.
- Pour arrêter, `docker compose down`.

1.6. Outillage, installation d'OWASP ZAP

OWASP ZAP est un outil gratuit d'analyse de sécurité des applications web. Tu vas l'utiliser tout au long de la mission pour explorer l'application et détecter automatiquement certaines failles.

- Télécharge OWASP ZAP depuis le site officiel `zaproxy.org` et installe-le.
- Au premier lancement, choisis le mode automatique pour découvrir l'interface.
- Lance une analyse automatique, appelée *Automated Scan*, sur l'adresse `http://localhost:8080`, et observe la liste des alertes remontées. Tu y reviendras à chaque étape pour confirmer tes trouvailles manuelles.

Tes premières observations avec ZAP :

2. Le référentiel OWASP Top 10

L'OWASP Top 10 est le référentiel de référence des dix risques de sécurité les plus critiques pour les applications web. Les équipes sécurité s'en servent comme grille d'audit. Sa version 2021 comporte les catégories suivantes, que tu vas auditer une par une sur le périmètre confié.

Code	Catégorie OWASP 2021
A01	Contrôles d'accès défaillants
A02	Défaillances cryptographiques
A03	Injection, qui inclut désormais le Cross Site Scripting
A04	Conception non sécurisée
A05	Mauvaise configuration de sécurité
A06	Composants vulnérables et obsolètes
A07	Identification et authentification de mauvaise qualité
A08	Manque d'intégrité des données et du logiciel
A09	Carence des systèmes de contrôle et de journalisation
A10	Falsification de requête côté serveur, en anglais SSRF

Chaque section qui suit est construite de la même façon. Une explication de la catégorie, une mise en pratique guidée sur l'application, puis un encadré où tu rédiges ton analyse et ta correction. Suis les étapes dans l'ordre.

3. A03, Injection SQL

3.1. Comprendre

Une injection survient lorsque des données fournies par l'utilisateur sont interprétées comme du code par le serveur. Dans le cas d'une injection SQL, l'attaquant insère des fragments de requête dans un champ de saisie, par exemple un formulaire de connexion. Si l'application construit sa requête en collant directement la saisie, l'attaquant peut modifier le sens de la requête, contourner une authentification ou voler des données.

3.2. Pratiquer

1. Sur la page d'accueil, dans le champ utilisateur, saisis `admin'` - avec n'importe quel mot de passe, puis valide. Observe le résultat.
2. Essaie ensuite `' OR '1'='1` dans les deux champs.
3. Avec ZAP, retrouve l'alerte correspondant à l'injection SQL et lis sa description.

3.3. Analyser et corriger

Explique pourquoi ces saisies fonctionnent. Puis décris la correction, qui repose sur les requêtes préparées avec des paramètres liés, où la saisie n'est plus jamais interprétée comme du code. Applique la correction dans le code et vérifie que l'attaque échoue.

Ton analyse et ta correction, A03 injection SQL :

4. A03, Cross Site Scripting, XSS

4.1. Comprendre

Le XSS est une autre forme d'injection, cette fois de code dans la page affichée au navigateur. Si l'application réaffiche une donnée fournie par l'utilisateur sans l'échapper, un attaquant peut y glisser du JavaScript qui s'exécutera chez les autres visiteurs, par exemple pour voler leur session. On distingue le XSS réfléchi, où le code revient dans la réponse immédiate, et le XSS stocké, où le code est enregistré puis servi à tous.

4.2. Pratiquer

1. XSS réfléchi, visite `http://localhost:8080/?nom=<script>alert(1)</script>` et observe.
2. XSS stocké, connecte-toi, va sur le forum interne, et publie un message contenant `<script>alert('xss')</script>`. Recharge la page.
3. Repère les alertes XSS dans ZAP.

4.3. Analyser et corriger

Explique la différence entre les deux types observés et leur dangerosité respective. La correction consiste à échapper systématiquement toute donnée utilisateur à l'affichage. Applique-la et vérifie.

Ton analyse et ta correction, A03 XSS :

5. A01, Contrôles d'accès défaillants

5.1. Comprendre

Cette catégorie, classée numéro un par l'OWASP, regroupe tous les cas où un utilisateur peut accéder à des ressources ou des actions qui ne devraient pas lui être permises. Cela inclut l'accès à des pages réservées, la consultation des données d'autrui, ou l'élévation de privilèges d'un simple utilisateur vers un compte administrateur.

5.2. Pratiquer

1. Connecte-toi avec le compte `stagiaire`, un simple utilisateur.
2. Tente d'accéder directement à `http://localhost:8080/admin.php`.
3. Constate que le panneau d'administration et la liste des comptes s'affichent.

5.3. Analyser et corriger

Explique pourquoi cet accès ne devrait pas être possible. La correction impose de vérifier, côté serveur et sur chaque page sensible, que l'utilisateur possède bien le rôle requis. Applique le contrôle et vérifie qu'un simple utilisateur est désormais refusé.

Ton analyse et ta correction, A01 :

6. A07, Identification et authentification de mauvaise qualité

6.1. Comprendre

Cette catégorie concerne les faiblesses des mécanismes de connexion, par exemple l'absence de limite sur le nombre de tentatives, des mots de passe triviaux autorisés, ou une gestion de session négligée. Elle facilite les attaques par force brute et le vol de comptes.

6.2. Pratiquer

1. Observe que l'application autorise un mot de passe aussi faible que `admin123` pour le compte administrateur.
2. Constate qu'aucune limite de tentatives n'existe sur le formulaire de connexion, ce qui rend une attaque par force brute possible.

6.3. Analyser et corriger

Décris les mesures attendues, à savoir une politique de mots de passe robustes, une limitation du nombre de tentatives, et idéalement une authentification renforcée. Propose au moins une correction concrète et applique-la.

Ton analyse et ta correction, A07 :

7. A02, Défaillances cryptographiques

7.1. Comprendre

Cette catégorie couvre la mauvaise protection des données sensibles, par exemple des mots de passe stockés en clair, l'absence de chiffrement des échanges, ou des secrets codés en dur dans le code source. La conséquence est qu'en cas de fuite, les données sont immédiatement exploitables.

7.2. Pratiquer

1. Via le panneau d'administration atteint plus tôt, observe que les mots de passe sont affichés et stockés en clair.
2. Ouvre le fichier de configuration et repère le secret codé en dur dans le code.

7.3. Analyser et corriger

Explique le risque. La correction repose sur le hachage des mots de passe avec une fonction dédiée, et sur le fait de ne jamais stocker de secret en clair dans le code. Applique le hachage et vérifie.

Ton analyse et ta correction, A02 :

8. A05, Mauvaise configuration de sécurité

8.1. Comprendre

Une mauvaise configuration regroupe les réglages dangereux laissés en place, par exemple un mode debug actif en production, des messages d'erreur trop bavards, ou des pages techniques exposées au public. Ces éléments donnent à l'attaquant des informations précieuses pour préparer son attaque.

8.2. Pratiquer

1. Visite `http://localhost:8080/status.php` et observe les informations exposées.
2. Note tout ce qu'un attaquant peut apprendre, version, moteur, mode debug.

8.3. Analyser et corriger

Explique pourquoi exposer ces informations est risqué. La correction consiste à désactiver le mode debug, à masquer les détails techniques, et à ne pas exposer de page de statut publique.

Ton analyse et ta correction, A05 :

9. A06, Composants vulnérables et obsolètes

9.1. Comprendre

Une application repose sur des bibliothèques, des frameworks et des moteurs qui vieillissent. Quand un composant n'est plus à jour, des vulnérabilités publiques peuvent exister et être exploitées. Savoir identifier les versions utilisées et leur état de support est une compétence clé.

9.2. Pratiquer

1. Sur la page de statut, relève la version du moteur de base de données annoncée comme obsolète.
2. Cherche si cette version est encore supportée et si des vulnérabilités connues lui sont associées.

9.3. Analyser et corriger

Explique le risque d'utiliser un composant obsolète. La correction consiste à maintenir un inventaire des composants, à suivre les annonces de sécurité et à mettre à jour régulièrement.

Ton analyse et ta correction, A06 :

10. A04, Conception non sécurisée

10.1. Comprendre

Cette catégorie vise les défauts présents dès la conception, indépendamment de la qualité du code. Un exemple typique est une fonctionnalité pensée sans contrôle de sécurité, comme un téléversement de fichier qui n'a jamais prévu de restreindre les types acceptés.

10.2. Pratiquer

1. Va sur la page de téléversement de document depuis ton espace.
2. Téléverse un fichier de type quelconque, y compris un fichier qui ne devrait pas être accepté, et observe l'absence totale de contrôle.

10.3. Analyser et corriger

Explique pourquoi un téléversement sans restriction est dangereux, notamment le dépôt de fichiers exécutables. La correction impose une liste blanche d'extensions, une vérification du type réel et une limite de taille.

Ton analyse et ta correction, A04 :

11. A08, Manque d'intégrité des données et du logiciel

11.1. Comprendre

Cette catégorie concerne le fait d'accepter des données ou des mises à jour sans vérifier leur intégrité ni leur origine. Désérialiser une donnée fournie par le client sans contrôle, ou appliquer une mise à jour non signée, expose l'application à des manipulations graves.

11.2. Pratiquer

1. Examine la page de mise à jour de configuration de l'application.
2. Constate qu'elle accepte une donnée sérialisée fournie par l'utilisateur et l'applique sans aucune vérification.

11.3. Analyser et corriger

Explique le danger de faire confiance à une donnée non vérifiée. La correction consiste à ne jamais désérialiser une entrée non fiable, et à vérifier signatures et intégrité avant d'appliquer toute donnée ou mise à jour.

Ton analyse et ta correction, A08 :

12. A10, Falsification de requête côté serveur, SSRF

12.1. Comprendre

Le SSRF survient lorsqu'une application va chercher une ressource à une adresse fournie par l'utilisateur, sans valider cette adresse. L'attaquant peut alors forcer le serveur à contacter des ressources internes normalement inaccessibles, par exemple des services locaux ou des points de métadonnées d'un hébergeur cloud.

12.2. Pratiquer

1. Depuis ton espace, utilise la fonction d'aperçu d'une URL externe.
2. Remplace l'URL par une adresse interne, par exemple `http://localhost` ou une adresse locale, et observe que le serveur tente de la joindre pour toi.

12.3. Analyser et corriger

Explique pourquoi laisser le serveur joindre une adresse arbitraire est dangereux. La correction repose sur une liste blanche stricte des destinations autorisées et le rejet des adresses internes.

Ton analyse et ta correction, A10 :

13. A09, Carence des systèmes de contrôle et de journalisation

13.1. Comprendre

Sans journalisation des événements de sécurité, une intrusion peut passer totalement inaperçue. Cette catégorie souligne l'importance d'enregistrer les tentatives de connexion, les accès sensibles et les erreurs, puis de surveiller ces journaux pour détecter une attaque.

13.2. Pratiquer

1. Sur la page de statut, observe que la journalisation des accès est annoncée comme désactivée.
2. Réfléchis à ce qu'un attaquant pourrait faire sans laisser de trace exploitable.

13.3. Analyser et corriger

Explique pourquoi l'absence de journaux est un problème, en particulier pour l'analyse après incident. Propose une journalisation des événements sensibles, par exemple les échecs de connexion, et une surveillance régulière.

Ton analyse et ta correction, A09 :

14. Scénario final, démonstration d'impact

Niveau expert. À traiter après avoir audité toutes les catégories précédentes.

14.1. Le scénario

La Direction Technique te demande une démonstration d'impact à présenter en restitution. Une faille isolée impressionne peu un décideur. Ce qui marque, c'est une chaîne d'attaque, c'est à dire l'enchaînement de plusieurs failles qui, mises bout à bout, mènent à une compromission totale. Ta mission est de construire et de documenter cette chaîne, de la première faille jusqu'au contrôle complet de l'application.

14.2. Objectif à atteindre

Partir d'un simple visiteur sans compte, et parvenir à obtenir l'ensemble des comptes et de leurs mots de passe, puis à déposer sur le serveur un fichier de ton choix, et enfin à prouver que le serveur peut être détourné pour contacter une ressource interne. Tu dois réaliser cet enchaînement en t'appuyant sur au moins quatre des catégories OWASP étudiées, et expliquer à chaque maillon pourquoi il fonctionne et comment il aurait été évité.

14.3. Contenu attendu dans cette partie du rapport

1. Un schéma ou une description ordonnée de ta chaîne d'attaque, maillon par maillon.
2. Pour chaque maillon, la faille OWASP exploitée, la manipulation exacte, et une capture d'écran.
3. Une analyse de l'impact global pour l'entreprise si cette chaîne était exploitée par un véritable attaquant.
4. Un plan de remédiation priorisé, c'est à dire l'ordre dans lequel tu corrigerais ces failles et pourquoi.

14.4. Critère de réussite

Le défi est réussi si ta chaîne est cohérente, reproductible à partir de ton rapport, qu'elle mobilise réellement plusieurs failles distinctes, et que ton plan de remédiation est argumenté. La qualité de la rédaction compte autant que la prouesse technique.

Ta chaîne d'attaque et ton plan de remédiation :

15. Prolongements professionnels

Si tu as terminé et que tu veux approfondir, explore la documentation officielle de l'OWASP sur chacune des catégories, et entraîne-toi sur des plateformes légales et dédiées à l'apprentissage, conçues pour être attaquées sans risque juridique. Ne teste jamais ces techniques en dehors d'un cadre explicitement autorisé.